

Caracteres parámetros de un Script

\$0 nombre del script

\$1 primer parámetro del script

\$2 segundo parámetro del script

...

\$n n-ésimo parámetro del script

\$# número de parámetros del script

\$* lista de parámetros del script

Sentencia if

- Se utiliza **if** en el intérprete de comandos (bash) para ejecutar una serie de operaciones si (**o si no**) se cumple una condición booleana

```
if expresiónBooleana
```

```
then
```

```
    sent1
```

```
    ...
```

```
    sentn
```

```
else
```

← *opcional*

```
    sent1
```

```
    ...
```

```
    sentn
```

```
fi
```

Sentencia if

- Ejemplo *script* usando if

vi siMiNotaEs

```
if [ $1 -ge 5 ]          <- test $1 -ge 3
then
    echo ' Has aprobado :-)'
else
    echo ' Has suspendido :-( '
fi
```

\$bash siMiNotaEs 7

Sentencia case

- Se utiliza **case** en el intérprete de comandos (bash) para ejecutar una serie de operaciones si se cumple una condición de tipo cadena de caracteres.

case *expresión* **in**

opción1)

sent1

...

sentn;;

opción2)

← *opcionales*

sent1

...

sentn;;

...

opciónn)

sent1

...

sentn;;

← en la última opción no es obligatorio ;;

esac

Sentencia case

- Ejemplo *script* usando case

vi palabraAgudaAcabadaEn

```
case $1 in
    a|e|i|o|u|n|s)
        echo Al acabar en vocal en n o en s
        echo LLEVA TILDE;;
    *)
        echo Al no acabar ni en vocal ni en n ni en s
        echo NO LLEVA TILDE
esac
```

\$bash palabraAgudaAcabadaEn u

Sentencia for

- Se utilizar **for** en el intérprete de comandos (bash) para realizar tareas repetitivas

```
for variable in lista  
do  
    sent1  
    ...  
    sentn  
done
```

```
for variable in lista ; do sent1 ; ... sentn ; done
```

Sentencia for

- Ejemplo *script* usando for

vi dimeLasVocales

¿Cómo sería con case en lugar de if?

```
for valor in $*
do
    if ([ $valor = a ] || [ $valor = e ] || [ $valor = i ] ||
        [ $valor = o ] || [ $valor = u ])
    then
        echo $valor es una vocal
    fi
done
```

\$bash dimeLasVocales a r t n s u p e

Sentencia while

- Se utilizar **while** en el intérprete de comandos (bash) para realizar tareas repetitivas siempre que se cumpla una expresión booleana

```
while expresiónBooleana
```

```
do
```

```
    sent1
```

```
    . . .
```

```
    sentn
```

```
done
```

```
while expresiónBooleana ; do sent1 ; ... sentn ; done
```


Sentencia while

- Ejemplo *script* usando while

vi decrementa

```
valor=$1
while [ $valor -gt 0 ]
do
    echo $valor
    valor=$((expr $valor - 1))      <- valor=$((valor-1))
done
```

expr evalúa expresiones aritméticas

\$bash decrementa 7

Caracteres parámetros de un Script

`$*` lista de parámetros del script

`"$*"` todos los parámetros forman una única cadena de caracteres

Diferencia `$*` y `"$*"`

vi dimeCuantosElementos

```
for cadena1 in $*  
do  
    echo $cadena1  
done  
  
echo ---  
  
for cadena2 in "$*"  
do  
    echo $cadena2  
done
```

`$bash dimeCuantosElementos e1 e2 e3 e4 e5`

Funciones en un *script*

vi ejemFuncScript

```
nombreFuncion1() {      o también      function nombreFuncion1{      <- definición
    sent1
    ...
    sentn
}
...
nombreFuncionX() {      o también      function nombreFuncionX{      <-definición
    sent1
    . . .
    sentm
}
...
nombreFuncion1          <- invocación sin parámetro
...
nombreFuncionX par1 ... parL      <- invocación con L parámetros
...
```

\$bash ejemFuncScript

Funciones en un *script*

vi calculaSumas

```
suma() {  
    v=$(( $1+$2 ))  
    echo La suma de $1 y $2 da $v  
}  
  
suma 3 4  
suma 2 2  
suma 1 6
```

`<- v=$(expr $1 + $2)`

\$bash calculaSumas

Funciones en un *script*

vi calculaSumasYtotal

```
suma() {  
    v=$(( $1+$2 ))                                <- v=$(expr $1 + $2)  
    echo La suma de $1 y $2 da $v  
    tot=$(( tot+v ))                               <- tot=$(expr $tot + $v)  
}  
  
dimeTotal() {  
    echo La suma de todo es $tot  
}  
  
tot=0  
suma 3 4  
suma 2 2  
suma 1 6  
dimeTotal
```

\$bash calculaSumasYtotal